

Final Project Report
Spring 2026 CMPE 3815
Grant Parks & Harrison Costa

Opening, Context, & Structure

Intro/Overview

This project builds on our previous motor control lab by adding motion based input using accelerometers. An Arduino reads hand movements from the accelerometer and converts them into control signals for our car, allowing the user to drive it through simple gestures like tilting and rotating the hand. The goal is to demonstrate how sensor data can be processed and mapped to real time control of a physical system in a simple and intuitive way.

Equipment List

Car Equipment List:

- Arduino board
- Computer & USB cable for programming/communication
- 4x DC motors + motor control shield
- 4x Mecanum wheels
- Various other hardware: breadboard, wires, resistors, etc,
- Mechanics: robot platform, batteries, battery holder, etc.

Hand Control Equipment List:

- Arduino board
- Computer & USB cable for programming/communication
- Miscellaneous Wires
- Bluetooth Modules
- Accelerometers
- Various other hardware: breadboard, wires, resistors, etc,
- Mechanics: platform, batteries, battery holder, etc.

Project Definition

For our final project, we are using an Arduino with accelerometers mounted on a hand to control the car wirelessly or through direct input. The car responds to hand motion, so tilting the hand forward, backward, left, or right controls the car's movement. The goal is to create a simple hand gesture controlled vehicle using sensor input, Arduino code, and the motor control system from the previous lab.

Technical Content & Methodology

Description of Approach

Our approach is to translate hand motion into real time control of our car using sensor data and an Arduino. Accelerometers mounted on the hand measure tilt and movement, which are read as acceleration values by the Arduino. These values are then interpreted into simple control commands such as forward, reverse, and turning. The processed signals are sent to the motor driver system, allowing the car to respond directly to hand gestures. This setup connects sensor input, signal processing, and motor control into one system that shows intuitive motion control. We approached it with a guess and check mentality and put our best effort forward and adapted if it did not perform as intended.

Sub-System Descriptions: Vehicle Movement Adaptation

This subsystem takes the processed motion inputs from the accelerometer and converts them into physical movement of the car. The Arduino sends control signals to a motor driver, which powers the motors to move the car forward, backward, and turn. This connects the sensor input to real world motion. Below, in *Figure 1*, a code snippet to help visualize the motor control is shown. The full code can be found at the bottom of our report in the *Code Appendix*. Also, below in *Figure 2* and *Figure 3*, a photo of the car, arduino and connections chart is shown.

```
// Motor driver pin definitions
const int ENA = 5; // PWM pin for Motor A speed
const int ENB = 6; // PWM pin for Motor B speed
const int IN1 = 7; // Motor A direction
const int IN2 = 8;
const int IN3 = 9; // Motor B direction
const int IN4 = 10;

void setup() {
  // Set all motor control pins as outputs
  pinMode(ENA, OUTPUT);
  pinMode(ENB, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
}

// Function to move forward
void moveForward() {
  digitalWrite(IN1, HIGH); // Motor A forward
  digitalWrite(IN2, LOW);
```

```

digitalWrite(IN3, HIGH); // Motor B forward
digitalWrite(IN4, LOW);

analogWrite(ENA, 200); // Set speed (0-255)
analogWrite(ENB, 200);
}

// Function to move backward
void moveBackward() {
digitalWrite(IN1, LOW);
digitalWrite(IN2, HIGH); // Reverse Motor A
digitalWrite(IN3, LOW);
digitalWrite(IN4, HIGH); // Reverse Motor B

analogWrite(ENA, 200);
analogWrite(ENB, 200);
}

// Function to turn left
void turnLeft() {
digitalWrite(IN1, LOW);
digitalWrite(IN2, HIGH); // Left motor backward
digitalWrite(IN3, HIGH);
digitalWrite(IN4, LOW); // Right motor forward

analogWrite(ENA, 180);
analogWrite(ENB, 180);
}

// Function to turn right
void turnRight() {
digitalWrite(IN1, HIGH);
digitalWrite(IN2, LOW); // Left motor forward
digitalWrite(IN3, LOW);
digitalWrite(IN4, HIGH); // Right motor backward

analogWrite(ENA, 180);
analogWrite(ENB, 180);
}

// Function to stop the car
void stopCar() {
analogWrite(ENA, 0);
analogWrite(ENB, 0);
}

void loop() {
// This loop will later receive commands from accelerometer logic
// For now, you can manually call functions to test movement

moveForward();
delay(2000);

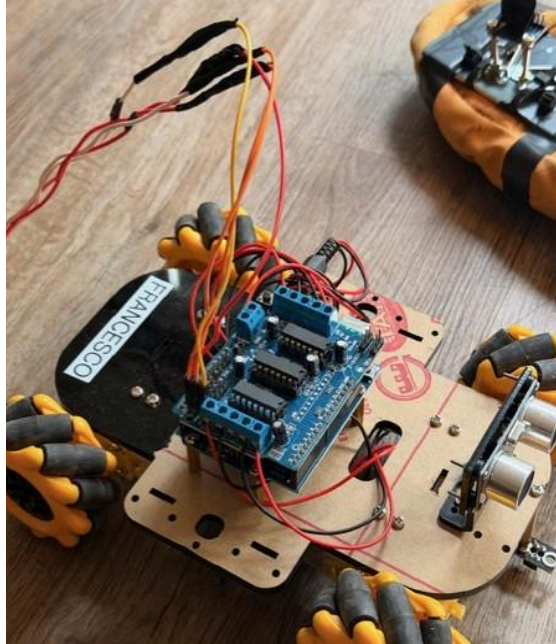
stopCar();
delay(1000);

moveBackward();
delay(2000);

stopCar();
delay(1000);
}

```

(Figure 1: Visualizing motor control)



(Figure 2: View of car and arduino set up)

Arduino -> Car

A5 -> SDA Pin
A5 -> SCL Pin
11 -> Motor driver PWM2A / M1 speed
3 -> Motor driver PWM2B / M2 speed
6 -> Motor driver PWM0A / M3 speed
5 -> Motor driver PWM0B / M4 speed
4 -> Motor driver DIR_CLK
7 -> Motor driver DIR_EN
8 -> Motor driver DATA
12 -> Motor driver DIR_LATCH
5V -> Motor driver logic power
GND -> Motor driver GND
Battery pack -> Motor power input

(Figure 3: Connections Chart of Arduino to car)

Testing:

This subsystem was tested by manually calling each movement function (forward, backward, left, right) and observing the car's response. Motor direction and wiring were adjusted as needed to ensure correct movement. Speed values (PWM) were tuned to get smooth and controlled motion. Once verified, this subsystem was ready to be integrated with the accelerometer input for full "hand" control.

Sub-System Descriptions: Movement Sensing Elements

This subsystem uses accelerometers mounted on the user's hand to detect motion and orientation. The sensors measure acceleration in multiple axes (typically X and Y), which are used to determine tilt direction. These values are read by the Arduino and serve as the input for controlling the car's movement. Below, in *Figure 4* a snippet of code shows how we put in a “dead zone” so that when the hand control was near level, but not perfect, the car would not be moving. The full code can be found at the bottom of our report in the *Code Appendix*. Also below, in *Figure 5* and *Figure 6*, a photo of our accelerometer mount and the connections chart for it are shown.

```
// Define accelerometer input pins
const int xPin = A0; // X-axis (left/right tilt)
const int yPin = A1; // Y-axis (forward/back tilt)

// Variables to store readings
int xVal = 0;
int yVal = 0;

// Define center (neutral) values – these may need to be tuned
int xCenter = 512;
int yCenter = 512;

// Dead zone range (prevents small movements from triggering the car)
int deadZone = 50;

void setup() {
  Serial.begin(9600); // Start serial monitor for debugging
}

void loop() {
  // Read accelerometer values
  xVal = analogRead(xPin);
  yVal = analogRead(yPin);

  // Print values for testing
  Serial.print("X: ");
  Serial.print(xVal);
  Serial.print(" Y: ");
  Serial.println(yVal);

  // Check if within dead zone (hand is level → no movement)
  if (abs(xVal - xCenter) < deadZone && abs(yVal - yCenter) < deadZone) {
    Serial.println("STOP (Dead Zone)");
  }
  // Forward (tilt one direction)
  else if (yVal > yCenter + deadZone) {
    Serial.println("FORWARD");
  }
  // Backward (tilt opposite direction)
  else if (yVal < yCenter - deadZone) {
    Serial.println("BACKWARD");
  }
  // Right turn
  else if (xVal > xCenter + deadZone) {
    Serial.println("RIGHT");
  }
  // Left turn
  else if (xVal < xCenter - deadZone) {
    Serial.println("LEFT");
  }

  delay(100); // Small delay for stability
}
```

(Figure 4: Code snippet of “dead zone”)



(Figure 5: Photo of accelerometer wired up to car and mounted to control glove)

Arduino Pin	MPU9255
5V	VCC
GND	GND
A4	SDA
A5	SCL

(Figure 6: Connections chart of accelerometer to arduino)

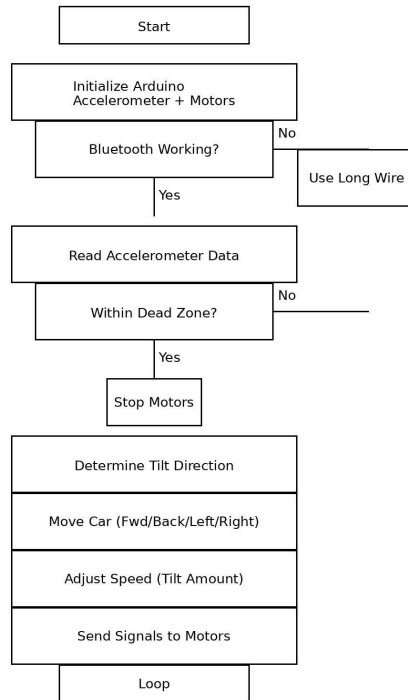
Testing:

The subsystem was tested by monitoring real time movements of the car while it was wired together, without including bluetooth yet, while tilting the hand in different directions. Threshold ranges were identified for each motion (forward, backward, left, right). Noise in the signal was reduced by averaging readings and adjusting delay times. This ensured stable and repeatable motion detection before integrating with the motor control system.

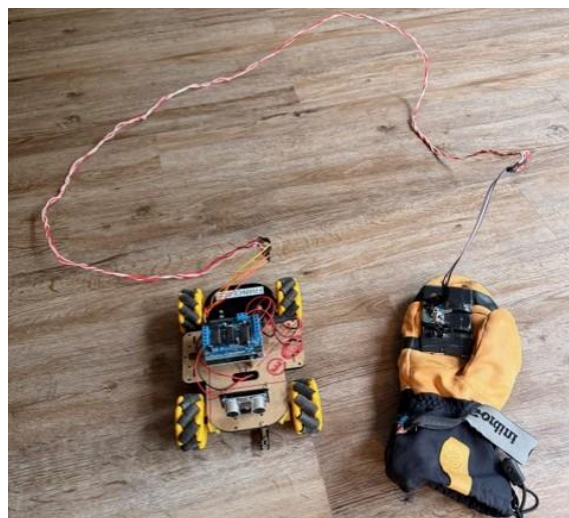
Overall approach: Putting the systems together

The overall decision making process for the car control system is shown in *Figure 7* and a complete photo of our project in *Figure 8*. *Figure 7* presents a flowchart of the system logic, starting with initialization of the Arduino, accelerometer, and motor driver, followed by checking the communication method. We originally tried to use bluetooth to communicate between the hand controller and the car, however our transmitter was accidentally broken in the process. We switched to a 5 foot long hardwired connection with 20 AWG stranded wire from Lowe's to ensure performance. The Arduino then reads accelerometer

data and applies a dead zone to prevent unintended movement when the sensor is near level. If the tilt exceeds this threshold, the direction and magnitude of motion are determined and mapped to motor commands, which are sent to drive the car accordingly. *Figure 8* shows the full physical setup used to implement this system, including the Arduino, accelerometer, wiring, and car all connected by wire.



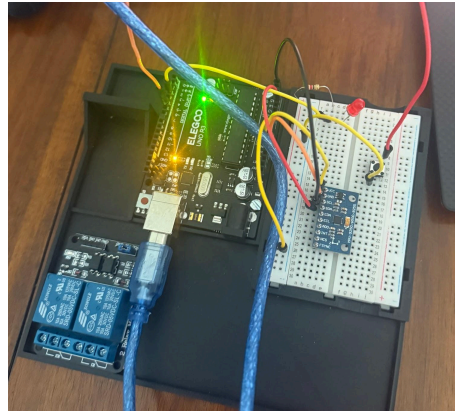
(Figure 7: Flow chart showing car decision making)



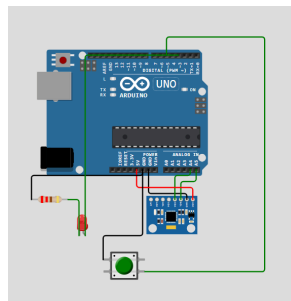
(Figure 8: Photo of complete wired project setup)

While diagnosing and testing the accelerometer before implementing it into our car project we made a test circuit. We put it on a breadboard connected to a light and a button. The light would change brightness depending on how many degrees of tilt there was and the button would reset the “zeroed position”. This

helped us implement the accelerometer controls into the car controlling project. A photo of the circuit we made can be seen below in *Figure 9* as well as a diagram of the circuit in *Figure 10*. The full code for this setup can be found at the bottom of this report in our *Code Appendix*.



(Figure 9: Photo of accelerometer testing circuit)



(Figure 10: Circuit diagram of accelerometer testing circuit)

Performance Commentary

The system performs reliably and responds accurately to hand motion using the accelerometer. The dead zone prevents unintended movement, and the car consistently moves in the correct direction with smooth speed control based on tilt. While Bluetooth communication was initially attempted, it did not function consistently, leading to constant connection failure. After switching to a wired connection, performance improved significantly and the system now operates as intended. The system now works effectively with the wired setup, though Bluetooth control could be revisited in the future with different equipment. Shown below, in *Figure 9*, is the original bluetooth setup before realizing it would not correctly work a viable percentage of the time and switching it to a wired connection as shown above in *Figure 8*.



(Figure 11: Bluetooth transmitter setup that was later abandoned)

Conclusion/Reflection

This project showed how hand motion can be used to control a vehicle using simple sensors and an Arduino. The accelerometer provided real time input, which was turned into movement commands for the car. By adding a dead zone and tuning the controls, the system became stable and easy to use. Overall, the project combined sensor input, signal processing, and motor control to create a responsive and easy to use motion controlled car. A key challenge was unreliable Bluetooth communication, which required redesigning the system to use a wired connection. This showed the importance of adaptability and testing in real engineering systems.

Full Code Appendix

Code for Accelerometer (including “zeroing button” and light indicator)

```
#include <MPU9255.h> // include MPU9255 library

float Ax, Ay, Az;

const int LED = 3;
unsigned long lastblink = 0;
bool ledstate = false;

int button = 5;
int buttonval = 0;

float zeroffset = 0.0;

MPU9255 mpu;

void setup() {
```

```

Serial.begin(115200);

if (mpu.init()) {
    Serial.println("initialization failed");
} else {
    Serial.println("initialization successful!");
}

mpu.set_acc_scale(scale_2g);

pinMode(button, INPUT_PULLUP);
pinMode(LED, OUTPUT);
}

void loop() {

    // Read accelerometer
    mpu.read_acc();

    // Read button
    buttonval = digitalRead(button);

    Serial.println(buttonval);

    // Convert raw values to g's
    Ax = (float)mpu.ax / 16384.0;
    Ay = (float)mpu.ay / 16384.0;
    Az = (float)mpu.az / 16384.0;

    // Print values
    Serial.print("AX: "); Serial.print(Ax, 3); Serial.print(" g");
    Serial.print(" | AY: "); Serial.print(Ay, 3); Serial.print(" g");
    Serial.print(" | AZ: "); Serial.print(Az, 3); Serial.println(" g");

    // Zero button
    if (buttonval == LOW) {
        zeroffset = atan2(Ax, Az) * (180.0 / PI);
        Serial.println("ZEROED!");
        delay(500); // debounce
    }

    // Calculate angle
    float angle = atan2(Ax, Az);
    float angle_degrees = angle * (180.0 / PI) - zeroffset;

    Serial.print("Angle: ");
    Serial.println(angle_degrees);

    // Use degrees for logic
    float absAngle = abs(angle_degrees);
    int blinkrate;

    // Corrected order
    if (absAngle > 100) {
        blinkrate = 300;
    } else if (absAngle > 50) {
        blinkrate = 100;
    } else if (absAngle > 25) {
        blinkrate = 60;
    } else if (absAngle > 1) {

```

```

    blinkrate = 10;
  } else {
    blinkrate = 1200;
  }

  // LED blink logic
  if (millis() - lastblink >= blinkrate) {
    ledstate = !ledstate;
    digitalWrite(LED, ledstate);
    lastblink = millis();
  }

  delay(250);
}

```

Code for our accelerometer controlled car

```

#include <MPU9255.h>
#include <Wire.h>

// Motor driver pins
const int PWM2A = 11;    // Motor 1 speed
const int PWM2B = 3;     // Motor 2 speed
const int PWM0A = 6;     // Motor 3 speed
const int PWM0B = 5;     // Motor 4 speed
const int DIR_CLK = 4;   // Shift register clock
const int DIR_EN = 7;    // Motor driver enable
const int DATA = 8;     // Shift register data
const int DIR_LATCH = 12; // Shift register latch

// Direction sent to the shift register
const int Move_Forward = 39; // Drive forward
const int Move_Backward = 216; // Drive backward
const int Left_Rotate = 106; // Spin left in place
const int Right_Rotate = 149; // Spin right in place
const int Stop = 0; // Stop all motors

// Accelerometer variables
MPU9255 mpu;
float Ax, Ay, Az;
float zerooffsetFB = 0; // Forward/backward calibration offset
float zerooffsetLR = 0; // Left/right calibration offset

int baseSpeed = 110; // Motor speed out of 225

// Sends speed and direction to all 4 motors
void Motor(int Dir, int s1, int s2, int s3, int s4) {
  analogWrite(PWM2A, s1);
  analogWrite(PWM2B, s2);
  analogWrite(PWM0A, s3);
  analogWrite(PWM0B, s4);
  // Send direction byte to shift register
  digitalWrite(DIR_LATCH, LOW);
  shiftOut(DATA, DIR_CLK, MSBFIRST, Dir);
  digitalWrite(DIR_LATCH, HIGH);
}

void setup() {
  Serial.begin(115200);
}

```

```

// Check MPU9255 is connected on I2C
Wire.begin();
Wire.beginTransmission(0x68);
byte error = Wire.endTransmission();
if (error == 0) {
    Serial.println("MPU found at 0x68");
} else {
    Serial.println("MPU NOT found - check wiring");
}

// Initialise MPU9255
if (mpu.init()) {
    Serial.println("MPU failed");
} else {
    Serial.println("MPU ready");
}
mpu.set_acc_scale(scale_2g); // Set to ±2G range

// Set motor driver pins as outputs
pinMode(DIR_CLK, OUTPUT);
pinMode(DATA, OUTPUT);
pinMode(DIR_EN, OUTPUT);
pinMode(DIR_LATCH, OUTPUT);
pinMode(PWM2A, OUTPUT);
pinMode(PWM2B, OUTPUT);
pinMode(PWM0A, OUTPUT);
pinMode(PWM0B, OUTPUT);
}

void loop() {
    // Read accelerometer and convert to G force
    mpu.read_acc();
    Ax = (float)mpu.ax / 16384.0;
    Ay = (float)mpu.ay / 16384.0;
    Az = (float)mpu.az / 16384.0;

    // Calculate tilt angles in degrees
    // Axes remapped to match VCC-forward mounting orientation
    float angleFB = atan2(-Ay, Az) * (180.0 / PI) - zerooffsetFB;
    float angleLR = atan2(-Ax, Az) * (180.0 / PI) - zerooffsetLR;

    Serial.print("FB: "); Serial.print(angleFB);
    Serial.print(" | LR: "); Serial.println(angleLR);

    int threshold = 25; // Ignore tilts smaller than 25 degrees
    int speedLeft = baseSpeed;
    int speedRight = baseSpeed;

    if (angleFB > threshold) {
        // Tilt forward - drive forward
        Serial.println("FORWARD");
        if (angleLR > threshold) {
            Serial.println("TURN RIGHT");
            speedRight = baseSpeed * 0.5; // Slow right side to curve
        }
        else if (angleLR < -threshold) {
            Serial.println("TURN LEFT");
            speedLeft = baseSpeed * 0.5; // Slow left side to curve
        }
    }
    Motor(Move_Forward, speedLeft, speedRight, speedLeft, speedRight);
}

```

```

}
else if (angleFB < -threshold) {
    // Tilt backward - drive backward
    Serial.println("BACKWARD");
    if (angleLR > threshold) {
        Serial.println("TURN RIGHT");
        speedRight = baseSpeed * 0.5;
    }
    else if (angleLR < -threshold) {
        Serial.println("TURN LEFT");
        speedLeft = baseSpeed * 0.5;
    }
    Motor(Move_Backward, speedLeft, speedRight, speedLeft, speedRight);
}
else if (angleLR > threshold) {
    // Tilt right only - rotate right in place
    Serial.println("ROTATE RIGHT");
    Motor(Right_Rotate, baseSpeed, baseSpeed, baseSpeed, baseSpeed);
}
else if (angleLR < -threshold) {
    // Tilt left only - rotate left in place
    Serial.println("ROTATE LEFT");
    Motor(Left_Rotate, baseSpeed, baseSpeed, baseSpeed, baseSpeed);
}
else {
    // No tilt detected - stop
    Serial.println("STOP");
    Motor(Stop, 0, 0, 0, 0);
}

delay(100); // Update 10 times per second

```